WILFRID LAURIER UNIVERSITY
LAURIER Inspiring Lives.

CP372 – Computer Networks (Winter 2026)

Assignment 3: Policy-Based Link State Routing

Due: Friday, March 27, 2026, at 11:59 pm
Language: Java
Late Policy: As stated in the course syllabus
Type: **Individual Assignment Constraint** (No group work or group submissions permitted)

1. Assignment 3: Policy-Based Link State Routing

In this assignment, you will implement a core functionality of the **Link State Intra-AS** routing algorithm for an autonomous system (AS) with a **POLICY ENFORCEMENT**. This autonomous system has a designated router (Security Agent, or SA) that performs special security processing on every datagram that leaves the AS.

POLICY ENFORCEMENT:

- Every datagram leaving the AS must pass through the SA **before** it reaches a gateway.
- Therefore, a gateway cannot be an intermediate node on the way to the SA.
- Additionally, the SA is the inspection point; once a packet leaves the SA, the first gateway it reaches should be its exit point.

You are given a weighted directed graph with **n** vertices (labeled with 1, 2, …, n) that represents the topology of a particular autonomous system. You are also given a list of "gateway routers" $x_1, x_2, \ldots, x_k$, where x_i are distinct numbers from the set {1, 2, …, n}. Finally, you know an integer from the set {1, 2, …, n}, which is the index of the router SA.

The output consists of **n-k** forwarding tables to gateway routers (one per non-gateway router), which allows datagram forwarding from a source router to a destination gateway AS routers along the shortest path, which must use an SA router as one of the intermediate points.

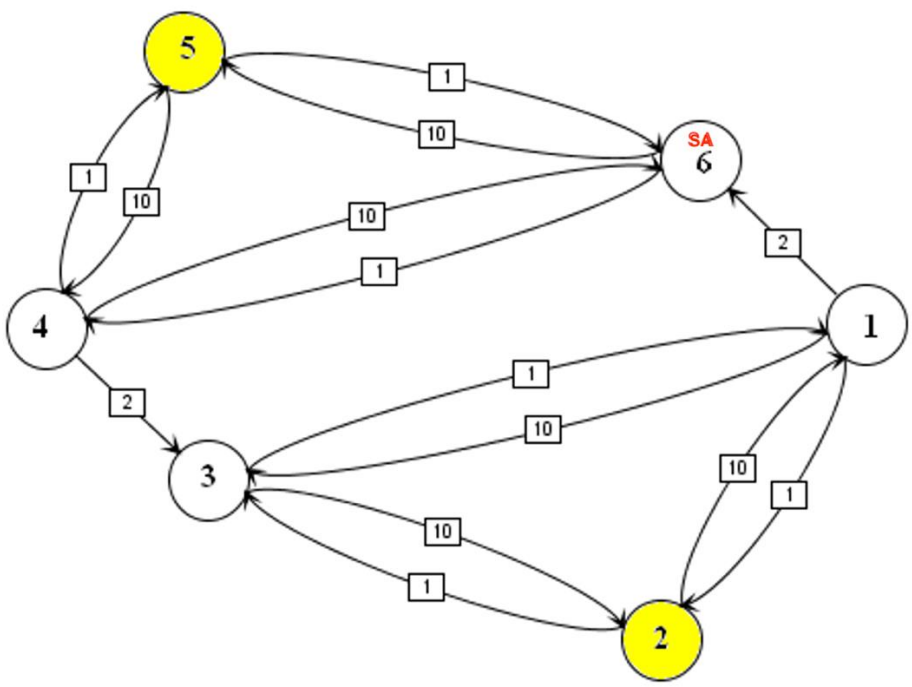
Your implementation must read input data from the system input and print the tables to the system output. The input starts with a single line containing the value of **n**, followed by n lines representing an adjacency matrix with weights, one more line representing a list of gateway routers, and the last line containing the index of SA, as in the following example:

```
6
0 1 10 -1 -1 2
10 0 1 -1 -1 -1
1 10 0 -1 -1 -1
-1 -1 2 0 1 10
-1 -1 -1 10 0 1
-1 -1 -1 1 10 0
2 5
6
```

Note:

- -1 represents an infinite weight (absence of directed edge)
- The matrix is not necessarily symmetric, as in the above example, where the link from router 1 to 3 has a weight of 10 and the backward link from router 3 to 1 has a weight of 1
- You can assume that gateway routers are listed in ascending order, also that the list is not empty, and **k<n**.

The sample input above corresponds to the following graph, with vertices 2 and 5 as gateway routers and vertex 6 as the SA.



The output for this input should consist of 4 forwarding tables (for routers 1,3,4,6) – each containing 2 entries that correspond to the shortest path to gateway routers. For example:

Forwarding Table for 1

To	Cost	Next Hop
2	6	6
5	4	6

Forwarding Table for 3

To	Cost	Next Hop
2	8	1
5	5	1

…

Forwarding Table for 6

To	Cost	Next Hop
2	5	4
5	2	4

Note that the route from 3 to 2 is 3->1->6->4->3->2 rather than 3->2 due to the policy.

Note additionally:

- Do not assume that every vertex is reachable from any other vertex (imagine graph with 2 connected components for example); if a particular gateway router is not reachable from the source print **-1** as the "Cost" and **-1** as the "Next Hop" destination; the same output is expected if there is no shortest path to a gateway router which goes through SA.
- You can assume correct input (no exception handling).
- Dijkstra's algorithm allows one to compute not only single-source shortest path weights, but also the predecessor for every node along the shortest path from the source, which is useful when constructing a forwarding table at the source.

Hint

Note that we are dealing with a directed graph G. Consider graph G', which is the transpose of graph G (the result of adjacency matrix transposition, which is the same as flipping the direction of all edges). You might find the following property useful: the shortest path P from x to y in graph G has the same weight as the shortest path P' from y to x in graph G' and uses the same intermediate vertices just in reverse order. Moreover, the predecessor of x in P' is the same as "next hop" from x in P. This makes it possible to compute necessary results in at most 2 calls to the standard Dijkstra algorithm.

What to submit:

- Single Java file RouteToGateway.java with your implementation.

The submitted Java file must use the default package (no package keyword).

It should be possible to compile your submission in a command-line environment with

```
> javac RouteToGateway.java
```

A bonus could be given for implementing "alternative forwarding": if there is an alternative path of the same weight from the source to a particular destination, it might be useful to find it as well. The entry in the forwarding table in this case might have two or more "next hop" elements (which is helpful for the alternative forwarding). Note that bonus points will be added to this assignment only.

Check the **RouteToGateway Testing Environment** [1] for an automated testing suite repository that includes various test cases to verify the Link-State routing implementation against the required security policy.

[1] https://github.com/MustafaDaraghmeh/RouteToGateway-TestingEnvironment

Test cases and expected outputs:

```
=====================================================
 TEST CASE: test_asymmetric.txt
=====================================================
[Input File Contents]
3
0 1 5
10 0 10
-1 2 0
2
3

[Execution Output]
-----------------------------------------------------
Forwarding Table for 1
To     Cost    Next Hop
2      7       3

Forwarding Table for 3
To     Cost    Next Hop
2      2       2
-----------------------------------------------------


=====================================================
 TEST CASE: test_bonus_alternative.txt
=====================================================
[Input File Contents]
5
0 10 10 -1 -1
-1 0 -1 5 -1
-1 -1 0 5 -1
-1 -1 -1 0 2
-1 -1 -1 -1 0
5
4

[Execution Output]
-----------------------------------------------------
Forwarding Table for 1
To     Cost    Next Hop
5      17      2,3

Forwarding Table for 2
To     Cost    Next Hop
5      7       4
```

```
Forwarding Table for 3
To     Cost    Next Hop
5      7       4

Forwarding Table for 4
To     Cost    Next Hop
5      2       5
-----------------------------------------------------


=====================================================
  TEST CASE: test_disconnected.txt
=====================================================
[Input File Contents]
4
0 1 2 -1
1 0 1 -1
2 1 0 -1
-1 -1 -1 0
2
3

[Execution Output]
-----------------------------------------------------
Forwarding Table for 1
To     Cost    Next Hop
2      3       3

Forwarding Table for 3
To     Cost    Next Hop
2      1       2

Forwarding Table for 4
To     Cost    Next Hop
2      -1      -1
-----------------------------------------------------


=====================================================
  TEST CASE: test_input.txt
=====================================================
[Input File Contents]
6
0 1 10 -1 -1 2
10 0 1 -1 -1 -1
1 10 0 -1 -1 -1
```

-1 -1 2 0 1 10
-1 -1 -1 10 0 1
-1 -1 -1 1 10 0
2 5
6
[Execution Output]

Forwarding Table for 1
To Cost Next Hop
2 7 6
5 4 6

Forwarding Table for 3
To Cost Next Hop
2 8 1
5 5 1

Forwarding Table for 4
To Cost Next Hop
2 10 3
5 7 3

Forwarding Table for 6
To Cost Next Hop
2 5 4
5 2 4

==========
TEST CASE: test_tie_break.txt
==========
[Input File Contents]
4
0 1 1 -1
-1 0 -1 2
-1 -1 0 2
-1 -1 -1 0
4
2 3

[Execution Output]

Forwarding Table for 1
To Cost Next Hop
4 3 2

```
Forwarding Table for 2
To     Cost    Next Hop
4      2       4

Forwarding Table for 3
To     Cost    Next Hop
4      -1      -1
-----------------------------------------------------


=====================================================
  TEST CASE: test_unreachable.txt
=====================================================
[Input File Contents]
3
0 5 2
-1 0 -1
-1 -1 0
2
3

[Execution Output]
-----------------------------------------------------
Forwarding Table for 1
To     Cost    Next Hop
2      -1      -1

Forwarding Table for 3
To     Cost    Next Hop
2      -1      -1
-----------------------------------------------------
```